

Continuăm cu o versiune mai bună: **TypeScript** va prinde unele erori la compilare, iar `verify()` va prinde erorile care depind de metadata: `retry`, `compensation`, `persistent tasks`.

1. Înlocuiește `src/core.ts`

⌄ TypeScript



```
export type Idempotency = "idempotent" | "non-idempotent";

export class Task<I, O> {
  public readonly retryEnabled: boolean;

  constructor(
    public readonly name: string,
    public readonly input: string,
    public readonly output: string,
    public readonly idempotency: Idempotency = "idempotent",
    public readonly persistent: boolean = false,
    public readonly compensation?: Task<any, any>,
    retryEnabled: boolean = false
  ) {
    this.retryEnabled = retryEnabled;
  }

  nonIdempotent(): Task<I, O> {
    return new Task(
      this.name,
      this.input,
      this.output,
      "non-idempotent",
      this.persistent,
      this.compensation,
      this.retryEnabled
    );
  }

  idempotentTask(): Task<I, O> {
    return new Task(
      this.name,
      this.input,
      this.output,
      "idempotent",
      this.persistent,
      this.compensation,
      this.retryEnabled
    );
  }

  persistentTask(): Task<I, O> {
    return new Task(
      this.name,
```

```

        this.input,
        this.output,
        this.idempotency,
        true,
        this.compensation,
        this.retryEnabled
    );
}

compensate(c: Task<O, I>): Task<I, O> {
    return new Task(
        this.name,
        this.input,
        this.output,
        this.idempotency,
        this.persistent,
        c,
        this.retryEnabled
    );
}

retry(): Task<I, O> {
    return new Task(
        this.name,
        this.input,
        this.output,
        this.idempotency,
        this.persistent,
        this.compensation,
        true
    );
}
}

export function task<I, O>(
    name: string,
    input: string,
    output: string
): Task<I, O> {
    return new Task<I, O>(name, input, output);
}

type OutputOf<T> = T extends Task<any, infer O> ? O : never;
type InputOf<T> = T extends Task<infer I, any> ? I : never;

export class Workflow<I, O> {
    constructor(public readonly tasks: Task<any, any>[]) {}
}

export function seq<A, B>(
    t1: Task<A, B>
): Workflow<A, B>;

```

```

export function seq<A, B, C>(
  t1: Task<A, B>,
  t2: Task<B, C>
): Workflow<A, C>;

export function seq<A, B, C, D>(
  t1: Task<A, B>,
  t2: Task<B, C>,
  t3: Task<C, D>
): Workflow<A, D>;

export function seq<A, B, C, D, E>(
  t1: Task<A, B>,
  t2: Task<B, C>,
  t3: Task<C, D>,
  t4: Task<D, E>
): Workflow<A, E>;

export function seq<A, B, C, D, E, F>(
  t1: Task<A, B>,
  t2: Task<B, C>,
  t3: Task<C, D>,
  t4: Task<D, E>,
  t5: Task<E, F>
): Workflow<A, F>;

export function seq(...tasks: Task<any, any>[]): Workflow<any, any> {
  return new Workflow(tasks);
}

export type VerificationError =
  | { kind: "ContractError"; message: string }
  | { kind: "RetrySafetyError"; message: string }
  | { kind: "CompensationError"; message: string };

export function verify(w: Workflow<any, any>): VerificationError[] {
  const errors: VerificationError[] = [];

  for (let i = 0; i < w.tasks.length - 1; i++) {
    const current = w.tasks[i];
    const next = w.tasks[i + 1];

    if (current.output !== next.input) {
      errors.push({
        kind: "ContractError",
        message: `${current.name} outputs ${current.output}, but ${next.name} expects ${n
      });
    }
  }
}

for (const t of w.tasks) {

```

```

    if (t.retryEnabled && t.idempotency === "non-idempotent") {
      errors.push({
        kind: "RetrySafetyError",
        message: `${t.name} is non-idempotent and cannot be retried safely`,
      });
    }

    if (t.persistent && !t.compensation) {
      errors.push({
        kind: "CompensationError",
        message: `${t.name} is persistent but has no compensation handler`,
      });
    }
  }

  return errors;
}

export function compileToASL(w: Workflow<any, any>): object {
  const errors = verify(w);
  if (errors.length > 0) {
    throw new Error(
      "Cannot compile invalid workflow:\n" +
        errors.map(e => `${e.kind}: ${e.message}`).join("\n")
    );
  }

  const states: Record<string, any> = {};

  w.tasks.forEach((t, i) => {
    const state: any = {
      Type: "Task",
      Resource: `arn:aws:lambda:REGION:ACCOUNT:function:${t.name}`,
    };

    if (t.retryEnabled) {
      state.Retry = [
        {
          ErrorEquals: ["States.ALL"],
          IntervalSeconds: 1,
          MaxAttempts: 3,
          BackoffRate: 2.0,
        },
      ];
    }

    if (i < w.tasks.length - 1) {
      state.Next = w.tasks[i + 1].name;
    } else {
      state.End = true;
    }
  });
}

```

```
    states[t.name] = state;
  });

  return {
    StartAt: w.tasks[0].name,
    States: states,
  };
}
```

2. Înlocuiește `src/examples.ts`

<> TypeScript



```
import { task, seq, verify, compileToASL } from "../core";

type CustomerRequest = { readonly tag: "CustomerRequest" };
type OrderCreated = { readonly tag: "OrderCreated" };
type OrderReserved = { readonly tag: "OrderReserved" };
type OrderPaid = { readonly tag: "OrderPaid" };
type OrderShipped = { readonly tag: "OrderShipped" };

const createOrder = task<CustomerRequest, OrderCreated>(
  "CreateOrder",
  "CustomerRequest",
  "OrderCreated"
);

const releaseInventory = task<OrderReserved, OrderCreated>(
  "ReleaseInventory",
  "OrderReserved",
  "OrderCreated"
);

const reserveInventory = task<OrderCreated, OrderReserved>(
  "ReserveInventory",
  "OrderCreated",
  "OrderReserved"
)
  .persistentTask()
  .compensate(releaseInventory);

const chargeCard = task<OrderReserved, OrderPaid>(
  "ChargeCard",
  "OrderReserved",
  "OrderPaid"
).nonIdempotent();

const shipOrder = task<OrderPaid, OrderShipped>(
  "ShipOrder",
  "OrderPaid",
  "OrderShipped"
```

```

);

const goodWorkflow = seq(
  createOrder,
  reserveInventory,
  chargeCard,
  shipOrder
);

console.log("Good workflow errors:");
console.log(verify(goodWorkflow));

console.log("Generated ASL:");
console.log(JSON.stringify(compileToASL(goodWorkflow), null, 2));

// Uncomment this to see TypeScript reject the workflow:
//
// const badTypestateWorkflow = seq(
//   createOrder,
//   shipOrder,
//   chargeCard
// );

const badRetryWorkflow = seq(
  createOrder,
  reserveInventory,
  chargeCard.retry(),
  shipOrder
);

console.log("Bad retry workflow errors:");
console.log(verify(badRetryWorkflow));

```

3. Rulează

⌵ Bash



```
npx ts-node src/examples.ts
```

Aici avem deja două lucruri importante pentru paper:

1. `seq(...)` oferă verificare statică a compoziției pentru workflow-uri secvențiale.
2. `verify(...)` oferă verificări de nivel workflow: retry safety și compensation.

Următorul pas este să adăugăm **generator de benchmark-uri + error injection**, ca să produci cifre reale pentru secțiunea Evaluation.